

What Is a Context Window?

Think of Claude's memory like a scrolling window. It can only see a fixed number of words at a time. As the conversation grows, old messages fall off the top. Claude Code handles this by compacting - summarizing old messages so less space is used, but some detail is lost.

A context window is measured in tokens (roughly 0.75 words per token):

200K context = ~150,000 words (about 300 pages)

1M context = ~750,000 words (about 1,500 pages)

Current Claude Models & Context Windows

Model	Context Window	Max Output	Input \$/1M	Output \$/1M
Claude Fable 5	1M tokens	128K	\$10.00	\$50.00
Claude Mythos 5	1M tokens	128K	\$10.00	\$50.00
Claude Opus 4.8	1M tokens	128K	\$5.00	\$25.00
Claude Opus 4.7	1M tokens	128K	\$5.00	\$25.00
Claude Opus 4.6	1M tokens	128K	\$5.00	\$25.00
Claude Sonnet 4.6	1M tokens	64K	\$3.00	\$15.00
Claude Haiku 4.5	200K tokens	64K	\$1.00	\$5.00

Note: Context windows do NOT change by plan. What changes by tier is rate limits (requests per minute).

The Full Optimization Ladder

Level 1 - Basic Solutions (No Code Needed)

Method	How It Works	Problem
Start fresh chats	New topic = new chat	You lose all previous context
Manual summarize	Copy summary into new chat	You do it manually every time
Keep chats short	One task per conversation	Not practical for long projects
Use files	Store notes in .md files and re-paste	Tedious, easy to forget

Level 2 - What Claude Code Does (Built-in Memory System)

Claude Code has a built-in memory system stored in files under:

C:\Users\Administrator\.claude\projects\...\memory\

Key facts are written to memory files. MEMORY.md is an index that is always loaded at the start of every chat. So even after a fresh session, Claude reads your memory and picks up context.

Limitation: MEMORY.md has a 200-line limit. It stores key facts, not everything.

Level 3 - Intermediate: RAG (Retrieval Augmented Generation)

Instead of putting everything in context, you store knowledge in a searchable database. When you ask a question, only the RELEVANT pieces are pulled out and given to the AI.

Like a library - you don't carry all books in your head, you look up what you need.

```
User asks: "What was the bug in pin_extractor?"
|
Search database for: "pin_extractor bug"
|
Find relevant chunk -> inject only that into context
|
AI answers with just that piece - no context wasted
```

Level 3b - Chunking + Summarization

Break large documents into small chunks. Summarize each chunk. Only load summaries unless full detail is needed.

Rolling Summary (what Claude Code compaction does):

1. Summarize the first half of the conversation
2. Drop the original messages
3. Keep only the summary + recent messages
4. Repeat as the conversation grows

Level 4 - Advanced: Vector Databases

Every piece of information is turned into a mathematical fingerprint called an embedding. When you ask something, your question's fingerprint is compared to all stored fingerprints. The closest matches are retrieved.

Popular tools: Pinecone, Weaviate, pgvector, ChromaDB

```
"Tell me about the poem function"
|
Convert to vector: [0.23, 0.87, 0.12, ...]
|
Find nearest stored vectors in database
|
Return: pin_extractor.py, poem variable, split() explanation
```

Level 4b - Knowledge Graphs

Store information as connected nodes - like a mind map in a database. Great for projects where things relate to each other.

```
poem          -> has method  -> split()
split()       -> returns    -> list
list          -> used in    -> pin_extractor
pin_extractor -> returns    -> secret_codes
```

Level 5 - Expert: Fine-tuning & Agent Memory

Fine-tuning: Train the AI model itself on your project's knowledge. The knowledge is baked into the model weights - no context needed at runtime. Expensive but permanent.

External Memory Stores (what production agents use): Agents can read and write to external systems - databases, files, APIs - as they work. They don't need to hold everything in context because they can look it up on demand.

Summary - The Full Ladder

```
BASIC      -> Start new chats / manual notes
|
BUILT-IN   -> CLAUDE.md + memory files (what we use now)
|
SMARTER    -> RAG - only pull relevant chunks into context
|
ADVANCED   -> Vector databases + embeddings
|
EXPERT     -> Knowledge graphs + fine-tuning + agent memory
```

For Your Python Training Project Specifically

Right now you have:

- Memory files storing key facts (your project, what you're learning)
- Compaction kicking in when the chat gets long (summarizes oldest parts)

If this were a LIVE PRODUCT (like a customer support bot that learns forever), you'd use:

RAG + a vector database - the most practical production solution that scales to infinite knowledge without ever hitting context limits.

The AI never "knows" everything upfront - it just knows where to look.